设计可扩张的 AI 开发流水线

AI Coding Agent 实战工作坊 — 进阶场 (Claude + Codex + Gemini)

目标： 从「管一间厨房」升级到「管多产线工厂」

John Wu | Apr 2026

今天的旅程

时段	做什么
观念讲解	Agentic 设计思维 + 治理边界
Live Demo	示范多 Agent 并行 review 流水线
回顾 Q&A	重点整理 + 课后练习

> 今天的关键字：**设计流水线**，不是亲自操作每一步

Part 1: 观念讲解

从「开餐厅」到「开发流水线」

中阶（上一场）	进阶（今天）
餐厅：一间厨房怎么运作	工厂：多条产线怎么协调
单一 Agent + 规则	多 Agent 编排 + 项目级 orchestration
Prompt / Context / Harness	Agentic Workflow 设计

> 上次你学会开一间厨房，今天你要设计整座工厂的流水线

三大支柱：进阶视角

支柱	中阶	进阶
Prompt	指令松紧度	多 Agent 派工
Context	模块化 CLAUDE.md	Playbook 分层（roles/stacks）
Harness	权限 + 反复调校	Hooks（tool-call 层硬挡）

> 从「会用」升级到「会设计」— 今天逐一展开这些进阶用法

Vibe Coding：什么时候该、什么时候不该？

适合 Vibe	该用更有结构的方式
即丢脚本 / 一次性爬虫	性能关键路径、安全相关 code
独立小功能、utility	跨多系统集成
探索性 prototype	架构决策（AI 不知道你的取舍）

> Vibe Coding 不是不好 — **判断何时该放手、何时该拿 spec 收敛**

Spec-Driven Development (SDD)

Vibe 不够时，先写 spec 再让 AI 做

开发方式	你做什么	AI 做什么
Vibe	说一句话	猜你要什么
SDD	**先写清楚 spec**	**照 spec 精准做**

一份 spec 至少回答三题：

What（要做什么）、**Constraints**（不能做什么）、**Done**（怎样算完成）

> Spec 越清楚，AI 产出越可预测、越可验收
> 后面的 OpenSpec `proposal.md` 就是在回答这三题

为什么需要工作流框架？

从 Prompt Engineering → Workflow Engineering

痛点	你一定遇过
上下文漂移	长对话后，AI 逐渐忘记前面的约束，质量退化
需求失真	需求只活在对话里，缺乏结构化沉淀
团队难复制	个人 prompt 技巧无法推广成团队能力
执行不稳定	大型重构、多文件联动，缺乏稳定推进机制

> 核心转变：从「提示词怎么写」→「**工作流怎么搭**」

Agentic Workflow — 你设计流水线，AI 跑产线

层次	比喻	做法
传统	手工作坊	你写所有 code
AI 辅助	你操作机器	你做事，AI 帮忙
Agentic	**你设计流水线**	**你设计流程，AI 自动执行**

你 = 流水线设计师

> **你** 设计流程（spec）+ 品管验收（review）
> **AI** 负责每一个工站（plan → implement → test）

多 Agent 架构 — 为什么要拆？

问题：一个 Agent 做太多事 → 质量下降

```
❌ 一个 Agent，又改前端又改后端又写测试 + 超长对话
   → Context 爆炸 → AI 忘记前面指令 → 质量崩坏
```

解法：每个子 Agent 专注一件事

```
✅ 主 Agent（生产线总监）
   ├── Agent A：只做前端，加载对应 stack + skills
   ├── Agent B：只做后端，加载对应 stack + skills
   └── Agent C：只做测试，加载对应 skills
```

> 跟微服务一样：**单一职责、接口明确**

多 Agent 的核心收益

不只是「并行加速」，更是 context 卫生

收益	说明
降低 context 密度	每个 session 只带必要信息
单一职责	每个 Agent 只专注一种任务
明确集成点	review 与验收有清楚边界

> 多 Agent 首先是 **context hygiene**（上下文干净），其次才是速度

什么时候用多 Agent？

适合	不适合
任务已有 plan，可拆明确子题	还在 brainstorm、问题没收敛
每个子题有清楚输入输出	任务太小，拆解成本 > 收益
有集成与 review 点	子题之间高度共享隐性 context

> 正确顺序是：**先 plan，再拆 Agent**

多模型协作：Claude Code 调度其他模型

透过 Plugin，Claude Code 可以调度其他模型

Plugin	调度谁	适合场景
codex-plugin-cc	OpenAI Codex	Code review、第二意见
cc-gemini-plugin	Google Gemini	大型 codebase 全局分析

- 个人感受：Codex 擅长规划，Claude 擅长实现，Gemini 擅长大范围分析

> 不同模型有不同强项 — 组合起来比单打独斗更稳

多模型分工：做事的和审查的要不同

阶段	做事的	Review 的
Plan	Codex	Claude
Coding	Claude	Codex
全局分析	Gemini	Codex / Claude

> **同一个模型不该自己做又自己审** — 换模型 = 换视角

Subagent 怎么被触发？

两种派工方式

方式	怎么触发	例子
自动派工	描述任务，主 Claude 自己挑 Agent	「做留言板，分角色并行开发」
手动指名	明确指定派哪个 Agent、用哪个模型	「派 code-reviewer 做 review」

> 每个子 Agent 独立 **context**，主 Claude 只看到最终回报

Subagent 也能指定模型

不只手动切，派工时就决定用哪个等级

```
请同时派两个 subagent 平行作业：

A（用 opus）：检查所有 prompts 是否符合 prompt-best-practices.md 的原则，
B（用 haiku）：把 README.md 的标题层级与条列符号统一成 playbook 其他 README 的风格
```

> 在流水线设计阶段就决定每个工站用什么等级的模型 — 不是事后手动切

Agent Handoff Contract

子 Agent 接什么、交什么，要写清楚

字段	要定义什么
责任范围	只负责哪一段工作
输入/输出	接手前拿到什么、完成后交付什么
可写范围	允许动哪些文件（例：reviewer 纯只读）
Done	验收条件 + 不可做的事

> 没有 handoff contract，就不是分工，而是把混乱分散出去

设定层级：项目级 vs 个人级

类型	项目级（跟着 repo 走）	个人级（跟着你走）
Prompts	`repo/CLAUDE.md`	`~/.claude/CLAUDE.md`
Settings	`repo/.claude/settings.json`	`~/.claude/settings.json`
Skills	`repo/.claude/skills/`	`~/.claude/skills/`

> 项目级由资深人员维护，个人级自己管理
> 冲突时**项目级优先** — 团队规范赢过个人偏好

为什么需要 Playbook？

一个团队、N 个项目，规范怎么不失焦

没有 Playbook	有 Playbook
每个 repo 自写 CLAUDE.md	共用一套，submodule 引入
新人各项目重读一次规范	认得一套就懂所有项目
规范更新手动推 N 个 repo	更新 Playbook，各 repo pull 即可

> Playbook = 团队共用的 AI 工作规范，跨项目一次落地

Playbook — 跨项目共用规范

用 `submodule` 统一管理

- playbook 是 **core + overlay**：
  `roles / stacks / policies / agents / hooks / skills / templates / guides`
- 所有项目引 submodule → 共用同一套规范

> 不用每个 repo 各写一份，**一处维护、多处生效**

Playbook 的分层结构

优先层级（高 → 低）+ 冲突时谁赢

```
CLAUDE.md                  ← 全局规则（最高）
  → policies/              ← 团队红线，例：安全规范、数据分级、PR 必审才能 merge
    → roles/               ← 角色视角
      → stacks/            ← 技术规则
        → skills/          ← 工作方法
          → templates/     ← 输入输出模板（最低）
```

> 当 Role 说的跟 Policy 冲突 → Policy 赢
> 当 Skill 说的跟 Stack 限制冲突 → Stack 赢

Permission Mode：你信任 AI 多少？

模式	AI 可以做什么	适合场景
default	多数操作都要你批准	高风险、不熟悉的任务
acceptEdits	文件编辑自动过，其他批准	日常开发
bypassPermissions	全部跳过权限询问	有充分护栏的自动化流程

> 信任程度要跟护栏强度匹配 — **越宽松，Hook 越不能少**

Agentic 常见失败模式

症状	根因	对策
Agent 偏离 plan	Spec 不够明确	补强 spec 的 constraints
子 Agent 产出互相冲突	缺少共享的接口约定	先定 API contract 再分工
越做越慢、质量下降	Context 被前面的错误污染	开新 session、带入干净文件

> 大部分 Agentic 失败不是 AI 不够聪明，而是 **spec 或 context 出了问题**

失败隔离与 Rollback — 让错误停在工站

做法	目的
每个子 Agent 独立 context	记忆不互相污染（可再搭 git worktree）
有副作用前先停下来等 review	挡住发 MR、动 DB 等不可逆操作
出错就重开干净 session	避免脏 context 继续累积

> 进阶不是避免出错，而是让错误 **可隔离、可回退、可重来**

Part 2: Live Demo — 多 Agent 流水线

主 Claude 实现 + 多个 reviewer 并行审查

加载 Playbook

初始化项目

```
mkdir live-demo && cd live-demo && git init
git submodule add https://github.com/johnwu1114/playbook.git
./playbook/setup.sh
```

> 一个命令把 **Prompt**、**Context**、**Harness** 三层规范同时落地
> ⚠️ 首次 `claude` 会跳 `@playbook/CLAUDE.md` approval dialog — **务必 approve**（误拒后 import 会停用，要手动 re-enable）

提需求

下指令给 Claude

```
做一个留言板，包含前端、后端、数据库用 docker 起。
设计完，分成三个角色并行开发，集成完跑起来给我看。

 (不要逐项问我规格，不加载 skill)
```

> 不指定技术栈，依 `@playbook/CLAUDE.md` 挑要加载哪个 stack / policy 覆盖

并行派 4 个 playbook reviewer

下指令给 Claude

```
并行派 code-reviewer / qa-checker / security-reviewer / migration-reviewer
review 刚刚的产出
```

多模型协作

下指令给 Claude

```
请 codex 做 adversarial-review；
若 codex 建议跨文件影响分析，再派 gemini 支援。
```

Hooks 何时触发

关键：hooks 挂在 Claude 的 tool call

`policies/agent-security.md` 写进 system prompt 让 AI 自律（soft）— demo 时 AI 根本不会下这些指令。

Hook 是最后一道硬防线：AI 真的尝试执行时才挡（hard）。

```
cat > .env <<< "OPENAI_KEY=sk-sample"   # secret-guard 挡
rm -rf ~/sample                         # dangerous-bash 挡
```

> Policy（prompt 层，靠 AI 自律）+ Hook（tool-call 层，硬挡）
> **两层都在，才叫 defense in depth**

Demo 回顾 — 每一步对应的支柱

Demo 步骤	做了什么	对应支柱
加载 Playbook	一行 setup 把 roles/stacks/policies 引入	**Context**
提需求	三角色并行实现（前端 / 后端 / DB）	**Prompt**
并行 review	4 reviewer 并行，不同视角审查	**Prompt**
多模型协作	Codex 做 adversarial，条件式升级 Gemini	**跨模型**
Hooks 何时触发	policy（prompt 层）+ hook（tool-call 层）	**Harness**

> **主 Claude 没有单打独斗**
> Playbook 提供知识、Agent 分工实现、reviewer 提供视角、Hook 守住底线

Part 3: 课后练习

把今天的方法套到你的工作场景

课后练习 — 打造你的 review 流水线

1. **在真实 MR 上试跑**
   `code-reviewer` + `qa-checker` + `security-reviewer`
2. **观察报告差异**
   谁提出「交叉验证建议」？有没有互相覆盖的维度？
3. **新增一个属于你的 Agent**
   例：`i18n-reviewer` / `perf-reviewer` / `api-contract-reviewer`
4. **把它放进团队 Playbook**
   好的 Agent 是团队资产，不是个人偏好

> 流水线的价值在**长期累积**：每多一站，整条线就多一层把关

两周落地路径

第 1 周	第 2 周
用内置 reviewer 跑 MR	新增一个自定义 reviewer
观察哪些建议重复	放进团队 playbook，邀人试跑
笔记常见误报	收 feedback，调 prompt

> 不要一次改整条流水线 — **先做一站，验证再扩**

Part 4: 回顾

怎么知道流水线有变好？ — Agentic 的观测指标

指标	它在反映什么
一次通过率	低了通常代表 spec 或 contract 不清楚
Review 打回率	高了代表交付质量不稳或验收点太晚
人工介入次数	高了代表自动化边界还不可信
Lead time	变长代表 handoff 太多或集成卡住
Token / task	上升通常代表 context 污染或拆太细；**多 Agent + 多模型时尤其需要关注**
Rollback 次数	高了代表风险控制设计得太后面

多 Agent 流水线的收益与成本

面向	说明
Context hygiene	每个 reviewer 独立 context — 不被实现对话污染
质量多维度	code / qa / security 各自专业视角，不会互相覆盖
并行省时	三站同时跑，比排队快 2–3 倍
团队资产累积	Agent `.md` 文件能共用、版本化、跨项目重用
维护成本	Agent prompt 要随团队规范演化，不能一次写完摆着
风险边界	reviewer 是建议者，最后决策仍在主 Claude / 人

> 流水线的价值在长期累积，但你还是要设计人为关卡与最终审核点

关键 Takeaway

1. 写的跟审的永远不同一个

主 Agent 实现，reviewer Agents 审查；独立 context 避免自我偏见

2. 每个工站的 prompt / model / 工具都为它的职责设计

sonnet 审逻辑、haiku 做对照题、**Gemini 吃 1M context 做跨文件影响分析**；reviewer 只给只读工具

3. 流水线的延伸性来自文件化的 Agent

一个 `.md` = 一个新工站；团队 playbook 让它们变成共享资产

> 进阶不是更会下 prompt — 是会设计让 AI 之间**互相把关**的流水线

三场工作坊回顾

场次	你学会的
初阶	体验 AI 协作，完成第一次可验收的协作
中阶	设计 AI 工作环境，让结果可重复
进阶	设计开发流水线，让 AI 替你跑流程

> 从「我用 AI」→「AI 稳定工作」→「AI 替我跑流程」

Q&A